Platform

```
npm install
npm run demo           # the data side: apps, installs, permissions, receipts
npm run demo:device    # the execution side: android, fan-out, repair queue, autom
npm run demo:modular   # slots, provider swap, model routing, harness, builder, im
npm run demo:surfaces # two MCP surfaces, the links page, availability + intent c
npm start              # HTTP API on :3000
```

`DATABASE_URL` switches it to real Postgres. Schema is `db/001_kernel.sql`, `db/002_workspace.sql` and `db/003_providers.sql`.

The split

Kernel is `src/` plus the two SQL files. It contains no business features. Everything else is a package in `modules/`, and every one of them is removable.

Packages come in kinds. Three matter so far:

- **`app`** — has its own data, a dashboard, and optionally a public section. QR menu, forms, CRM.
- **`channel_app`** — a real Android app the business installs on their own device and logs into. Facebook, Google Business Profile, Yelp. Carries an appmap.
- **automations** — data, not code. A trigger and a list of capability calls. Publishable.

Execution: the device, not an API

The executor is the business's own cloud Android instance. `provision()` creates two workloads on one host — the Android instance and a container — each with a persistent volume. The business installs real apps onto the Android instance and logs into them with their own accounts. The credential never leaves the workspace.

Each channel app ships an **appmap**: which canonical keys it carries, and for each key, the steps to write it and the steps to read it back.

```
"carries": ["hours", "contact.phone"],
```

```
"routes": {
  "hours": {
    "write": [{ "open": "com.facebook.katana" }, { "tap": "Hours" },
              { "type": "{{value.opens}}", "into": "opens_at" }, { "tap": "Save"
    "read":  [{ "open": "com.facebook.katana" }, { "tap": "Hours" },
              { "read": "opens_at" }, { "read": "closes_at" }],
    "assemble": { "opens": "opens_at", "closes": "closes_at" }
  }
}
```

`src/drivers/android.js` is the seam. Steps go in; a simulator runs them today, an ADB-backed cloud instance runs them later. Nothing above that file changes.

Fan-out: nobody says “update Facebook”

The owner changes hours once. `canonical.fact_changed` fires. `fanout.js` looks up every app installed on that device whose appmap declares it carries `hours`, and queues a `channel.push` against each one. The owner never picks destinations.

Real output from `demo-device.js`:

```
OWNER CHANGES HOURS ONCE, IN ONE PLACE
 owner set hours to 07:00 - 22:00. nobody told the platform where to put it.

 facebook          route android  succeeded  verification: verified
 google_business   route android  succeeded  verification: verified
 yelp              route android  succeeded  verification: verified
```

Verification is a screen read

A push is not believed because the steps ran. The verifier replays the appmap’s `read` steps, pulls the value off the screen, records it as that app’s own observation with an authority rank, and compares it to canonical. That comparison sets `in_sync`. A push that ran but did not land comes back `partial`, not `verified`.

When a map goes stale

Apps move things. When a step cannot find its target the run stops, writes a `repair_item` with the failing step and a screenshot, and fails. It does not guess at a nearby field and it does not report success:

```
YELP MOVES A FIELD (the map no longer matches the app)
```

```
result: failed, verification: failed
  repair queue: yelp — no field "from" on screen
                failing step: {"type":"07:00","into":"from"}
```

The live screen

`POST /api/b/:slug/device/session` opens a view or control session against the same instance the automations drive. Not a copy. `GET /api/b/:slug/device/screen` returns what is on it now.

Shared automations

One business writes an automation and publishes it. Another installs it by key — it holds a reference, not a copy. When the author publishes a new version, everyone who installed it gets the new behaviour without doing anything:

```
reef-charters installed "lead-to-crm" authored by ocean-view
  reef-charters contact: Cam Ruiz
after ocean-view published 1.1.0, with no action by reef-charters:
  deal: Inquiry from Priya Nandan (new)
```

Private stays private

Every business gets its own MCP surface at `/mcp/:slug`, built from the same projections that make the public profile. A package that did not declare a public section cannot appear on it. Its provisioned tables exist, are reachable by that app through the gateway, and are not on the surface.

Every layer is a slot

The kernel declares slots. A slot has a contract — the functions a provider must export. A provider package fills it. The kernel names no vendor anywhere.

```
workspace.executor   needs ["run","screenshot","prepare"]     one
model                needs ["models","complete"]               many
harness              needs ["run"]                             many
builder              needs ["plan"]                            one
memory               needs ["remember","recall"]               one
sandbox              needs ["test"]                            one
```

Swapping is one row in `provider_binding`. A business binding shadows the platform default. From `demo-modular.js`:

```
SWAPPING THE EXECUTION LAYER
 cloud executor: hours pushed, result {"packageKey":"facebook","key":"hours",...}
 rebound to: android_local_node  ({"latency":"lan","location":"on_premise"})
 pushes so far: verified, verified
 same appmaps, same capabilities, same receipts. one row changed where it runs.
```

The contract is checked when the package loads, not at 3am — a provider that does not export what its slot requires is rejected at boot.

Model routing

Model providers declare what they have and what each one costs. The router scores candidates against the task and falls back when one fails, recording that it did:

```
need=reasoning            -> hosted_models:reasoning-large
need=cheap                -> hosted_models:fast-small
need=vision               -> local_models:node-vision
need=reasoning local-only -> local_models:node-small

with the hosted provider unreachable:
need=reasoning            -> local_models:node-small
recorded: chose local_models:node-small, fell back from hosted_models:fast-small
```

`local: true` on a task means the data may not leave the workspace, and only providers whose models declare `local` are eligible.

A harness cannot widen its own reach

A harness is handed `invoke`, which goes through the same gate, approvals and receipts as a dashboard click. It is offered only capabilities the acting person could already invoke:

```
capabilities offered to the owner's harness: 8
capabilities offered to the staff harness:  1
withheld from staff: business.set_fact, business.set_hours, channel.push,
                     channel.read, business_profile.update_contact,
                     qr_menu.set_availability, qr_menu.remove_item
```

Builder: an intent becomes a package

```
intent: build a public song request board customers can scan and add a song titl
proposed package: public_song_request (app)
table: public_song_request fields ["name","song","title","artist"]
capabilities: .create .list .update .remove
state: proposed — nothing exists yet

approved. package public_song_request installed.
signature: valid, signer business:ocean-view, tier community
created a row through the new capability: verification verified
public profile sections now: About | Hours | Menu | Public Song Request | Contac
```

A manifest with a schema gets create/list/update/remove and a renderer for free (`src/kernel/generated.js`), each with a real verifier. The generated package goes through the same validator and the same installer as a hand-written one.

Import: open source becomes a package or it stops

Eleven steps, and the job row records which one failed and why:

```
fetch -> analyze -> graph -> license -> security -> pin
      -> sandbox -> capabilities -> permissions -> sign -> publish
```

```
example/waitlist -> published as waitlist
  license MIT, pinned 9f2c1ab, signed
example/copyleft -> rejected at license: AGPL-3.0 cannot be redistributed inside
example/greedy   -> rejected at permissions: declares capability outside its nam
```

A branch is not a package — the commit is pinned. Nothing installs unsigned; the signature covers the manifest hash, so changing what a package declares invalidates it.

Scripts edit. Models read.

A capability that writes into an outside app through the device is marked `agentSafe: false` . The harness is never offered it, and the private surface labels it `scriptOnly` . The appmap is the whole decision — no model chooses which button to press when something is being changed.

```
capabilities offered to the owner's harness: 7
withheld: channel.push (script-only)
script-only on the owner surface: channel.push
```

`channel.read` is `agentSafe: true`. A model may open an app and read it, because reading cannot change anything.

Two surfaces per business

```
ghost:circle-boats  (public)                    no credential
  ghost://circle-boats/about            from business_profile
  ghost://circle-boats/hours            from business_profile
  ghost://circle-boats/availability     from availability
  ghost://circle-boats/menu             from qr_menu
tools: read_about, read_hours, read_availability, read_menu, check_availability


ghost:circle-boats:private                      token bound to a membership
without a token: unauthorized
owner token -> acting as owner, 10 tools
staff token -> acting as staff, 2 tools
  staff sees: availability.search, availability.hold (asks)
```

The private surface is not a fixed tool list. It is built per token from what that membership is granted, so a tool that would be denied never appears, and a tool whose disposition is `ask` is labelled as one that will park for approval. `POST /api/b/:slug/tokens` issues one; it is shown once and stored hashed.

The links page

`GET /l/:slug` is a real page. Every tile is a projection from an installed app, and every one opens in place:

```
sections rendered: Hours | Check Availability | Menu
page size: 6.5 kb, zero outbound links to a booking platform
```

Install an app, a tile appears. Uninstall it, the tile goes. The open/closed badge is computed from canonical hours with any temporary closure layered on top — it is not typed into the page.

`GET /embed/:slug/:section` returns the same section as a fragment for the business's own website. One source, three places: links page, embed, MCP.

The search is the asset

Because the visitor never leaves, the business keeps what a booking platform would have

kept:

```
what the business now knows:
  links   2026-08-25  party 2  2 matched  booked
  links   2026-08-30  party 6  0 matched  did not book
```

The six-person request on the 30th found nothing. That is a demand signal, and it belongs to the business. `POST /public/:slug/availability/search` is unauthenticated and still recorded; `GET /api/b/:slug/intent` reads it back.

Rules the code enforces

- Grants, not roles. No grant means denied. Explicit `never` beats everything.
- A capability marked sensitive parks in `awaiting_approval` and writes an approval row.
- No verifier, no claim. A capability with no `verify` gets a receipt of `unknown`.
- Receipts hash-chain per business; `verifyChain` catches an edit or a deletion.
- Uninstall revokes reach, not data.
- No per-channel columns. What an app shows is an observation with an authority rank.
- A manifest cannot declare `business_id`, cannot claim a capability outside its namespace, and a channel app cannot claim to carry a key it has no route for.
- Apps compose through events, not imports.
- Device writes are scripts. `agentSafe: false` keeps them out of every model's hands.
- The private surface is derived from grants, never from a fixed tool list.
- Tokens are stored hashed and scoped to one membership at one business.
- A provider must export what its slot's contract requires, checked at load.
- A harness is offered only what the acting person may already invoke.
- Nothing installs unsigned, and the signature covers the manifest hash.
- An import that fails any gate produces a rejection with the step and reason, never a partially trusted package.

HTTP surface

```
GET    /api/packages                              what can be installed
GET    /api/capabilities                          every registered verb
GET    /api/b/:slug/installs                      what this business has installed
POST   /api/b/:slug/installs                      install one
DELETE /api/b/:slug/installs/:key                 uninstall one
POST   /api/b/:slug/invoke                        the only way to make anything happe
GET    /api/b/:slug/approvals                     what is parked waiting on a human
POST   /api/b/:slug/approvals/:executionId        approve or reject
GET    /api/b/:slug/receipts                      the ledger plus a chain check
GET    /api/b/:slug/drift/:key                    canonical vs every app

GET    /api/b/:slug/workspaces                    android instance and container
POST   /api/b/:slug/workspaces                    provision
GET    /api/b/:slug/device/apps                   apps on the device and login state
POST   /api/b/:slug/device/apps                   install an app and log in
DELETE /api/b/:slug/device/apps/:key              remove it
POST   /api/b/:slug/device/session                open the live screen
GET    /api/b/:slug/device/screen                 what is on it now
GET    /api/b/:slug/sync                          per app, per key, in sync or drifte
POST   /api/b/:slug/sync/:key                     force a fan-out
GET    /api/b/:slug/repairs                       maps that stopped matching

GET    /api/automations                           the automation marketplace
POST   /api/b/:slug/automations                   publish one
POST   /api/b/:slug/automations/:key/install      install one by key

GET    /api/slots                                 every slot and its contract
GET    /api/b/:slug/providers                     what is bound for this business
POST   /api/b/:slug/providers                     swap a provider
POST   /api/b/:slug/think                         route one task across model provide
GET    /api/b/:slug/routing                       what was chosen, and what it fell b
POST   /api/b/:slug/harness                       run an agent loop against a goal
GET    /api/b/:slug/harness                       past runs
POST   /api/b/:slug/build                         describe an app, get a plan
POST   /api/b/:slug/build/:planId/approve         deploy the plan as a package
POST   /api/b/:slug/build/:planId/reject          throw it away
GET    /api/import/steps                          the import pipeline
POST   /api/b/:slug/import                        bring a repo in
GET    /api/b/:slug/import                        past import jobs
GET    /api/packages/:key/:version/signature      verify a package signature

POST   /api/b/:slug/tokens                        issue a private-surface token
DELETE /api/b/:slug/tokens/:id                    revoke one
GET    /api/b/:slug/intent                        what visitors searched for

GET    /p/:slug                                   public profile as JSON
```

```
GET    /l/:slug                                the links page
GET    /embed/:slug/:section                   one section as an embeddable fragme
POST   /public/:slug/availability/search       unauthenticated, and recorded
GET    /mcp/:slug                              public surface
GET    /mcp/:slug/private                      private surface (Bearer token)
GET    /mcp/:slug/:section                     read one public resource
```

Providers shipped so far

package	fills	note
`android_cloud`	workspace.executor	persistent cloud instance
`android_local_node`	workspace.executor	same contract, on-premise hardware
`hosted_models`	model	declares models, cost, speed, strength
`local_models`	model	`local: true`, nothing leaves the workspace
`loop_harness`	harness	deterministic planner over allowed capabilities
`composer`	builder	intent to manifest

Apps shipped so far

package	declares	public section
`business_profile`	none; reads canonical	About, Hours, Contact
`qr_menu`	`menu_item`	Menu
`availability`	`slot`	Check Availability
`forms`	`form`, `submission`	Get in touch
`crm`	`contact`, `deal`	none
`facebook` / `google_business` / `yelp`	appmaps	none

Not built yet

The real ADB driver behind `src/drivers/android.js`. Login and credential capture on the device. Screen streaming and the control bridge — the session row and token exist, the transport does not. Screen reading by model instead of by appmap. Voice as an input. Real model endpoints — the providers declare and route correctly, but `complete()` simulates. A memory provider and a sandbox provider: the slots are declared, nothing fills them. Browser and desktop executors. Mesh. Dashboard and public UI; the API returns what they need. Consumer identity. Billing.